

AI 기반 개발 방법 사례

PDD + SDD + Harness + Feature Flag

30분

발표 시간

30슬라이드

전체 구성

4가지

핵심 방법론

사례 4선

실제 적용

이 발표자료는 어떻게 만들어졌나? 내용 작성 1.5시간 (Claude 챗봇)

1

 사용자

GarmentOS MES 개발계획 검토 질문
(실시간 수집, 20~30% 코드, KPI)

 기술 분석 답변
+ 개념 설명 + 코드 예시

2

 사용자

"PDD+SDD+Harness+FF 방법론으로
30분 세미나 자료 초안 만들어줘"

 25슬라이드 초안 제시
(스토리 구조 + 사례 조사 포함)

3

 사용자

"OpenClaw 기간 명시 / Cowork 보완
CC vs Codex 비교 슬라이드 추가
Harness Activity 중심으로 수정"

 28슬라이드 v2 전면 개정
(검색 기반 최신 사례 반영)

4

 사용자

"AI가 자동화 못하는 20~30% 설명해줘
개발자 KPI 방향도 제시해줘"

 5가지 유형 상세 분석
+ 3축 KPI 프레임워크 제안

5

 사용자

"위 내용을 부록 슬라이드로 정리하고
최종 30슬라이드 구성 확정해줘"

 부록 A·B 추가
최종 30슬라이드 목차 확정본 제시

6

 사용자

"PPTX 발표자료로 만들어줘"

 37슬라이드 PPTX 자동 생성
(섹션 브레이크 포함 · 1,197KB)

Claude 챗봇 : 총 대화 횟수: 약 5회 | 중간 검색: 5회 (실제 사례 사실 확인) | 최종 산출물: 37슬라이드 PPTX | 소요 시간: 1.5 시간

목 차

PART 1 슬라이드 1~3

세상이 바뀌었다

Stripe -96% · Wiz -99% · 현실을 직시하라

PART 2 슬라이드 4~5

기존 방식의 한계

버그 발견 비용 100배 · AI도 방법론이 없으면 동일

PART 3 슬라이드 6~10

4축 방법론 전체 지도

PDD · SDD · Harness ★ · Feature Flag

PART 4 슬라이드 11~16

Harness 심화

역할 분리 · H-SC-01 · H-RT-01 · 수치 검증

PART 5 슬라이드 17

도구 비교

Claude Code vs OpenAI Codex 철학 · 성능 · 적합 상황

PART 6 슬라이드 18~23

충격 실제 사례

GarmentOS · Stripe · OpenClaw · Cowork · Codex · 국내

PART 7 슬라이드 24~25

개발자 역할 재정의

코드 타이피스트 → 오케스트레이터 · 급별 역할

PART 8 슬라이드 26~28

마무리 + Q&A

3가지 즉시 액션 · "사람은 판단, AI는 실행"



부록 A

AI가 못 하는 20~30%의 정체 (5가지 유형)



부록 B

AI 시대 3가지 개발자 KPI 프레임워크

이것은 예외가 아닌 이미 다가온 현실

게임의 규칙이 바뀌었다

Stripe

글로벌 결제 인프라

-96%

소요 기간

Scala 10,000줄 → Java
수작업 예상 10주 → 4일 완료

Wiz

클라우드 보안

-99%

active coding time

Python 50,000줄 → Go
예상 수개월 → 20시간

Duolingo

교육 앱

+67%

PR 처리 속도 향상

PR 리뷰 주기 67% 단축
PR 처리량 70% 증가

AI에게 코드를 시켜봤더니...

문제는 AI가 아니고 방법론이 없었던 것이다

기대한 것

- ✓ AI가 알아서 만들어주겠지
- ✓ 한 번에 완성되겠지
- ✓ 버그가 없겠지
- ✓ AI가 알아서 고치겠지

현실

- ✗ 맥락 없는 범용 코드 쏟아짐
- ✗ 반복 수정 사이클 지속
- ✗ 도메인 특화 버그 신규 등장
- ✗ 동일 버그 다른 화면에서 재생산

PART 2

기존 방식의 한계

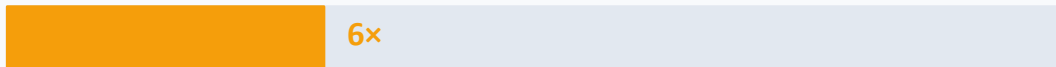
왜 지금 방법론이 필요한가?

늦게 발견된 버그, 100배 비용 초래

설계 단계



개발 단계



테스트 단계



운영 단계



① 해석 불일치

텍스트 요구사항 →
개발자마다 다른 해석

② 사후 테스트

테스트는 개발 완료 후 →
이미 늦었다

③ 버그 재발생

같은 유형의 버그가
다른 화면에서 재발

AI는 '빠르게' 잘못 만든다 — 방법론이 없으면

역설: 속도 증가 → 잘못된 방향의 코드가 더 빠르게 쌓인다.

① 컨텍스트 부재

"의류 OEM 공장"을 모르는 AI는 일반적인 공장 코드를 생성한다.

② 도메인 규칙 미반영

Bundle Race Condition 같은 현장 특화 버그를 모른다.

③ 테스트 없는 자신감

코드는 생성하지만 도메인 검증 테스트는 없다.

④ 오류 재생산

화면 A의 버그 패턴을 화면 B에서도 동일하게 반복한다.

AI를 잘 쓰려면 → AI에게 잘 시켜야 한다 → 방법론이 필요하다

PART 3

4축 방법론

PDD + SDD + Harness + Feature Flag

PDD + SDD + Harness + Feature Flag — 하나의 파이프라인



PDD

Prototype-Driven Discovery

"무엇을 만들까?"

화면으로 요구사항 확정
텍스트 해석 차이를 없앤다



SDD

Spec-Driven Development

"어떻게 만들까?"

코드 전에 Spec 완성
AI에게 정확한 지시서를 준다



Harness

도메인 결함 카탈로그

"무엇을 피할까?" ★

결함 패턴 사전 정의
AI가 구현 시 방어 코드 자동
생성



Feature Flag

기능 스위치

"실패하면 어떻게?"

5분 내 부분 롤백 스위치
두려움 없이 배포한다

결과: 재작업 80% 감소 | SIT 기간 50% 단축 | Claude Code 코드 생성 70~80% | 납기 리스크 최소화

PDD: 화면이 요구사항 — 텍스트 명세서 시대의 종말

핵심 아이디어

- ▶ 텍스트 요구사항은 사람마다 다르게 해석 가능
- ▶ 화면을 보여주면 피드백이 즉시·정확히 도출
- ▶ 오전 피드백 → 오후 Claude Code가 Spec 초안 생성
- ▶ 요구사항 확정 5일 → 1~2일로 단축

AI와의 연계

AI는 추상적 텍스트보다 구체적 Spec을 받을 때 훨씬 정확하게 만든다.

PDD는 AI를 위한 요구사항 확정 방법이기도 함

실제 적용 사례(GarmentOS)

1 HTML 프로토타입 34개 화면 선제작

AI가 화면 구조 자동 생성

2 고객이 직접 클릭하며 피드백

"이 버튼이 너무 작다", "이 컬럼 필요없다"

3 오후에 Claude Code가 Spec 초안 생성

오전 피드백 → requirements.md 자동 생성

4 MES 전문가 검토·보강

도메인 용어·계산 공식·예외 처리 추가

SDD: 코딩 전에 설계도가 먼저 — AI에게 정확한 지시서 제공

Spec = 화면 하나당 3개 파일

➤ requirements.md

기능 요구사항 + 계산 공식 + 예외 처리 + 인쇄 요구사항

➤ design.md

API 엔드포인트 + DB 테이블 + TypeScript 타입

➤ tasks.md

구현 순서가 포함된 태스크 목록

Claude Code 호출 방식 비교

✗ 나쁜 방법

"로그인 화면 만들어줘" → 범용 코드, 재작업 필수

✓ 좋은 방법 (SDD)

"requirements.md를 읽고 구현해줘.
Harness H-SC-01을 반드시 방어하고
단위 테스트도 함께 작성해줘."

SDD의 3대 효과



재작업 80% 감소

맥락이 있으니 AI의 첫 결과가 거의 정답



테스트 자동 도출

Spec의 예외 처리 → 단위 테스트로 자동 변환



Spec도 Git 관리

코드 롤백 시 Spec도 함께 자동 복원

Feature Flag: 켜고 끄는 스위치가 있으면 배포가 두렵지 않다

이 프로젝트 Flag 10개 (기능 레벨) 예시

Flag 이름	기본값	활성화 조건
cad-marker-import	false	CAD 파싱 구현 완료 후
sewing-iot-input	false	IoT 센서 연동 완료 후
erp-csv-import	false	ERP CSV 포맷 확정 후
sewing-sam-estimation	true	SAM 데이터 없으면 false
sewing-hourly-entry	true	운영 안정화 후 조정

Rollback 공식

git reset
+ Flag off 조합
→ 5분 내 복원

대형 회사 사례

- ▶ Facebook: 수천 개 Flag로 전체 코드 관리
- ▶ Netflix: 신기능을 1% 사용자에게 먼저
- ▶ Google: 대부분 신기능이 Flag=false로 배포

AI 시대에 특히 중요한 이유

Claude Code가 여러 화면을 동시에 만들 때 문제가 생기면
Flag 하나씩 끄며 격리 가능 → 원인 즉시 특정 가능

4축이 만드는 시너지 — 1 + 1 + 1 + 1 = 10 (GarmentOS)

구분	전통적 방식	4축 방법론 적용
요구사항 확정	텍스트 문서 (수 주)	PDD: 화면으로 즉시 확정
도메인 버그	테스트에서 발견 (10×비용)	Harness: 구현 시점에서 차단 (1×비용)
코드 재작업률	30~40%	5~10%
테스트 작성	별도 Task (시간 소요)	Harness 연동으로 자동 생성
배포 위험 대응	전체 롤백 (수 시간)	Flag: 기능 단위 5분 복원
SIT 기간	4주 (표준)	2주 계획 (Harness 효과)
AI 활용 방식	보조 도구	Spec 기반 메인 실행자 (70~80%)

29건

Claude Code 자동화 Task

32~42 인일

절감 추정 (Man-Day)

70~80%

AI 코드 생성 비율

2주

SIT 단축 목표

PART 4

Harness 심화

오류를 예방하는 현재까지 최신 방법론

결함은 예측 가능 - 사전 차단



도메인 패턴

버그의 70%는 해당 산업에서
반복되는 특정 패턴 안에 존재



AI 자동 방어

패턴을 먼저 정의하면
AI가 방어 코드+테스트를 자동 포함

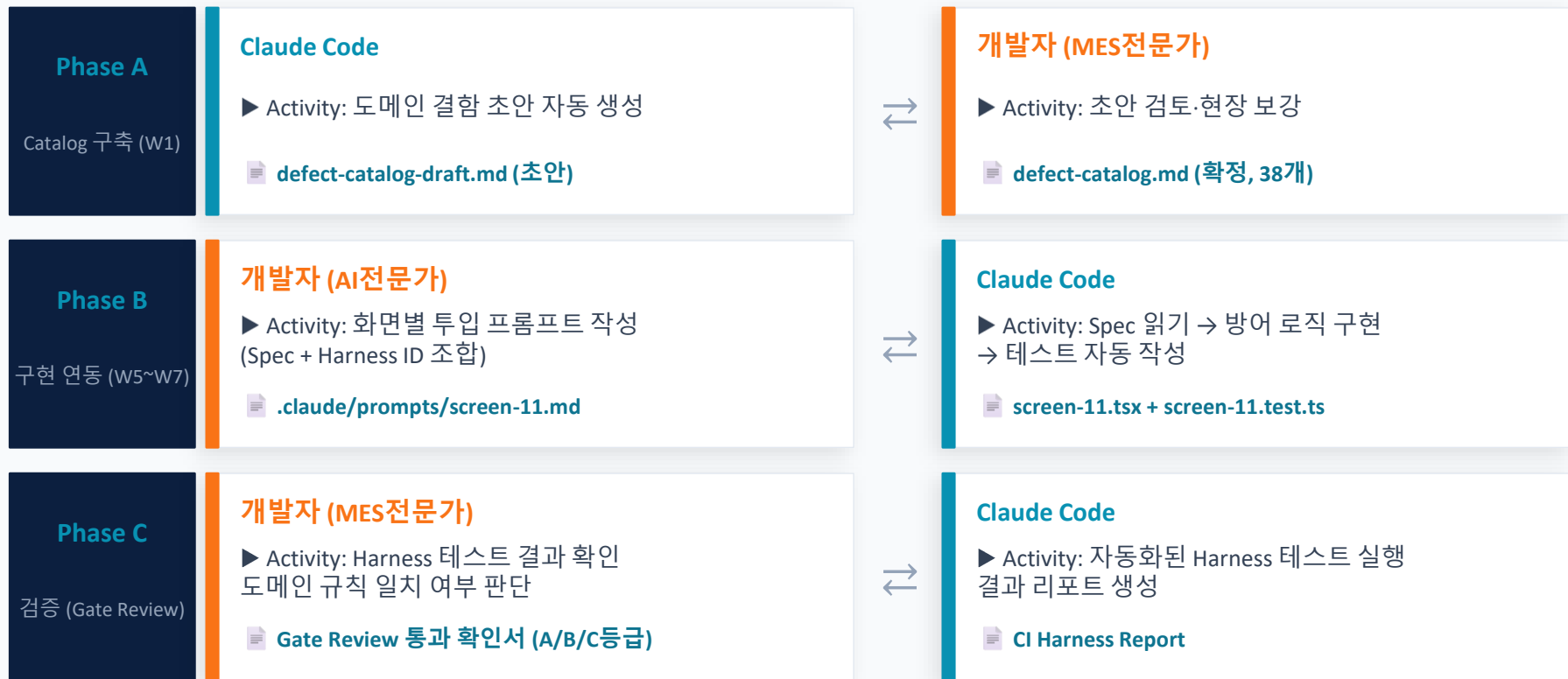


SIT 50% 단축

구현 시점에서 막으면
Phase 3 테스트 기간 절반 절약

TDD: 개발자가 테스트를 먼저 작성 | Harness: 도메인 전문가가 결함 패턴을 먼저 정의 → AI가 테스트+방어코드를 동시에 생성

Harness 운영 사이클 — 개발자와 Claude Code의 역할 분리 (GarmentOS)



사람이 결정: "무엇이 문제인가" (도메인 지식) | AI가 실행: "그 문제를 코드로 막는 방법" (구현 지식)

Harness 실전 사례 ① — H-SC-01: Bundle 번호 중복 (GarmentOS - Race Condition)

👤 개발자 (MES전문가) 작성

Catalog 기록 내용

결함 ID: H-SC-01

발생: 동시 Bundle 생성 → Race Condition

기존 코드: `SELECT MAX(bundle_no)+1 → INSERT`

→ 0.1초 차이로 두 사람이 같은 번호 획득

영향: LOT 추적 오염 / 재무 데이터 이상

방어: DB SEQUENCE 사용

Application 번호 생성 금지

🤖 Claude Code 자동 생성

방어 코드 + 테스트

```
-- 방어 코드 (H-SC-01)
CREATE SEQUENCE bundle_seq START 1;

async function createBundle(workOrderId) {
  // H-SC-01: DB SEQUENCE로만 생성
  return prisma.$queryRaw`
    INSERT INTO bundles (bundle_no, work_order_id)
    VALUES (NEXTVAL('bundle_seq'), ${workOrderId})
    RETURNING *`
}

// 자동 생성 테스트
test('H-SC-01: 동시 100개 생성, 중복 0건', async () => {
  const r = await Promise.all(
    Array(100).fill(null).map(() => createBundle('WO-001'))
  )
  const nums = new Set(r.map(b => b.bundle_no))
  expect(nums.size).toBe(100) // 중복 없음
})
```

개발자 Catalog 작성: 10분 → Claude Code 방어코드+테스트 생성: 3분 → Phase 3 SIT에서 이 버그 발견 비용: 제로

Harness 실전 사례 ② — H-RT-01: SAM 추정값 실측값 오인 (GarmentOS)

👤 개발자 (MES전문가) 작성

Catalog 기록 내용

결함 ID: H-RT-01

발생: data_source=SAM_ESTIMATE인 데이터가
[추정] 표시 없이 화면에 나타날 때

영향: 관리자가 추정값을 실측으로 오인
→ 라인 재배치 오판

방어: data_source 필드 필수 배지 표시
오차율 20% 초과 시 추정 일시 정지

🤖 Claude Code 자동 생성

배지 컴포넌트 + 테스트

```
// 방어 코드 (H-RT-01): 배지 필수 표시
function ProductionQtyBadge({ qty, dataSource }) {
  const isEstimate = dataSource === 'SAM_ESTIMATE'
  return (
    <div>
      <span>{qty}장</span>
      {isEstimate
        ? <Badge color="amber">[추정]</Badge>
        : <Badge color="green">[실측]</Badge>}
    </div>
  )
}

// 자동 생성 테스트
test('H-RT-01: SAM_ESTIMATE → [추정] 배지', () => {
  render(<ProductionQtyBadge qty={69}
    dataSource="SAM_ESTIMATE" />)
  expect(screen.getByText('[추정]')).toBeInDOM()
  expect(screen.queryByText('[실측]')).toBeNull()
})
```

개발자 결정: "실측/추정 구분 규칙" → Claude Code: 모든 실적 화면에 일괄 적용 → 관리 오판 위험 제거

Harness 카탈로그 전체 구조 — 38개 결함 패턴의 역할 분리(GarmentOS)

공정	결함 ID	결함 패턴	개발자 역할	Claude Code 역할
참고	H-WH-01	4점법 Roll 길이 0 → 나뉠셈 오류	조건 정의	입력 검증 로직 생성
참고	H-PRINT-01	프린터 오프라인 → UI 멈춤	에러 흐름 정의	에러 핸들러+재시도 생성
참고	H-PRINT-02	중복 라벨 인쇄 → 이중 계산	방지 규칙 정의	print_log 테이블 생성
재단	H-SC-01	Bundle 번호 중복 (Race Condition)	원인 분석	DB SEQUENCE 코드 생성
재단	H-SC-03	Marker 효율 원단 폭 미반영	공식 검증	파라미터 필수 검증 생성
봉제	H-SW-01	효율 분모(계획수량) 0 → NaN	예외 정의	0 처리 로직 자동 생성
봉제	H-SCAN-01	0.3초 내 중복 스캔	시간 기준 정의	디바운싱+로그 생성
QC	H-QC-01	DHU 계산 검사수량 0 → NaN	공식 정의	null 처리 로직 생성
QC	H-MFZ-01	MFZ 미검사 → 출하 법적 위반	법적 근거 확인	403 차단 API 생성
성능	H-PERF-01	WIP 전공정 N+1 쿼리 → 5초+	성능 기준 정의	인덱스+최적화 생성
실시간	H-RT-01	SAM 추정값 실측 오인	표시 규칙 정의	뱃지 컴포넌트 자동 생성

총 38개 패턴 | 사람이 결정: "무엇이 문제인가" | AI가 실행: "코드로 막는 방법"

Harness가 만들어내는 수치 — 구현 시점에서 차단 시 효과

Before — Harness 없음

Phase 3 SIT 기간: 4주

도메인 로직 버그

60%



화면 간 연동 버그

25%



성능 이슈

15%



← 이것들을 Harness가 미리 막는다

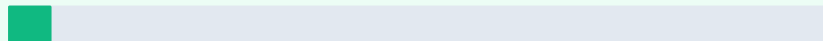
After — Harness 적용

Phase 3 SIT 기간: 2주 (50% 단축)

도메인 로직 버그

5%

← 구현 시점에 차단됨



화면 간 연동 버그

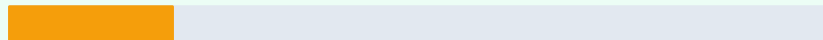
75%

← 이것만 남음



성능 이슈

20%



SIT 1건 수정 평균: 1일 (원인 명확)

4주→2주

SIT 기간 단축

4,000만원

4인팀 직접 비용 절감 (추정)

95%

도메인 버그 사전 차단

반나절

Catalog 초안 비용 (Claude Code 생성)

PART 5

도구 비교

Claude Code vs OpenAI Codex

Claude Code vs OpenAI Codex — 같은 AI 코딩 에이전트, 다른 철학

비교 항목	Claude Code (Anthropic)	Codex (OpenAI)
개발 철학	"두 번 재고, 한 번 자른다" Supervised Autonomy	"빠르게 실행한다" Unsupervised Autonomy
SWE-Bench 성능	72.5% (실제 GitHub 버그)	49% (동급 최신 기준)
자율성 방식	Plan Mode: 사전 검토 후 실행 사람이 변경 전 리뷰 가능	Full-Auto: 승인 없이 실행 태스크 보내고 자리 비울 수 있음
멀티에이전트	Agent Teams (tmux 기반) 팀장 에이전트 → 병렬 배분	Subagent 자동 생성 태스크별 자동 분기
Spec 친화성	CLAUDE.md + skills/ 파일 (SDD·Harness와 직결됨)	AGENTS.md 파일 (형식이 덜 정형화)
강점	복잡한 다파일 리팩토링 도메인 추론 + Harness 연동	CLI·배포·반복 작업 토큰 효율 (~3배 저렴)
적합 상황	Harness+SDD가 있는 팀 아키텍처 중심 프로젝트	범위 명확한 반복 태스크 CI 자동화, 배포 스크립트

실무 팁 (2026): "설계는 Claude Code, 단순 실행은 Codex" — 두 도구를 상황에 맞게 병용하는 팀이 증가 중

PART 6

실제 적용 사례

국내외 혁신 사례 4선+ 국내 적용

우리의 사례: GarmentOS MES — 4인 팀이 8인 규모 프로젝트 수행 (추정)

GarmentOS MES Phase 1 | 베트남 의류 OEM 봉제 공장(YIC) | 34개 화면 · 44개 API · 14주 · 4인 팀 + Claude Code

구분	전통적 방식 (추정)	4축 방법론 (실제 계획)	효과
팀 구성 필요	6~8인	4인 + Claude Code	팀 규모 50% 절감
예상 기간	20주 이상	14주 (계획)	납기 준수율 향상
요구사항 확정	3~4주 (문서 작성)	1~2주 (PDD 세션)	-50% 단축
코드 생성 비율	개발자 100%	Claude Code 70~80%	생산성 3~4배
자동화 Task	0건	29건 (32~42 인일 절감)	직접 비용 절감
Harness 결함 방어	없음	38개 패턴 사전 정의	SIT 도메인 버그 95% 차단
SIT 예상 기간	4주	2주 계획	납기 리스크 50% 감소

개발사 투자: Harness Catalog 초안 반나절 + CLAUDE.md 1일 + Spec 체계 정착 1~2주 → 전체 프로젝트 4주 단축 + 마진 개선

실제 사례 ① Stripe — 10 엔지니어·주 → 4일: 개발팀은 무엇을 경험했나

4일

vs
예상
10주

-96%

🔍 문제 상황

Scala 레거시 코드 10,000줄을 Java로 마이그레이션 필요.
"수작업으로 하기엔 너무 오래 걸려서 계속 미뤄온 작업."
이유: 더 새로운 JDK 지원 → 성능 개선 가능하지만 공수가 너무 큼

👥 역할 분리

Claude Code: 파일별 Scala→Java 변환 자동 수행
개발자: 변환 결과 Gate Review, JDK 호환성 검증, CI 통과 확인
→ 개발자는 "타이핑"이 아닌 "판단"에 집중

🔧 해결 방법

도구: Claude Code (기업용 서명 바이너리)
배포: 1,370명 전 엔지니어에게 zero-configuration 배포
보안 해결: Anthropic과 협력하여 서명된 기업용 바이너리 직접 제작
→ 일반 오픈소스 배포본은 기업 보안 정책 위반 문제를 이렇게 해결

💡 개발사 교훈

"계속 미뤄온 기술 부채를 4일에 해결했다"
"Vibes are up — 엔지니어 툴링 만족도 상승"
— Scott MacVicar (Stripe 개발 인프라 리더)
미루는 이유가 공수라면 → AI가 그 이유를 없앴다

실제 사례 ② OpenClaw — 1시간 프로토타입이 개발 생태계를 뒤흔들다.



OpenClaw 개발 타임라인

2025.11.14

아이디어 → 프로토타입 1시간

WhatsApp ↔ Claude Code CLI 연결 (접착 코드)

2025.11~12

개발자 커뮤니티 조용히 확산

GitHub Stars 2,000 개 (Clawdbot 이름)

2026.01.25

공개 런칭 → 하루에 9,000 Stars

"Stop chatting. Start doing." README

2026.01.27

Anthropic 상표권 경고 수신

"Clawdbot" → "Claude"와 유사, 48시간 내 변경

2026.01.30

보안 취약점 → OpenClaw로 재변경

CVE-2026-25253, 커뮤니티가 48시간 내 패치

2026.02

GitHub Stars 100,000개 돌파

역대 최단 기록 / Jensen Huang GTC 키노트 언급

1시간

최초 동작 프로토타입

100K

GitHub Stars (3개월)

1,000명

선전 사무실 설치 줄

개발자 교훈

- ▶ "가장 중요한 혁신은 더 똑똑한 AI가 아니었다"
- ▶ "언제 어디서나 접근 가능한 AI였다"
- ▶ 오류(CVE) 발생 시: 오픈소스 커뮤니티가 48시간 패치
- ▶ 창시자 Peter, OpenAI에 영입 (2026.02)

실제 사례 ③ Claude Cowork — OpenClaw이 증명한 것을 제품으로 완성

Claude Cowork (2026년 1월 Research Preview) + Dispatch (2026년 3월)

왜 만들었나

OpenClaw의 폭발적 성장이 증명:
"사람들은 AI에게 터미널 앞에서만
일을 시키고 싶지 않다"
→ Anthropic의 공식 대응

개발 도구·구성

개발 도구: Claude Code 자체 (dogfooding)
아키텍처: Electron 기반 데스크탑 앱
Dispatch: 모바일로 PC의 Claude 원격 제어
개발 기간: 베타버전은 1주 반, 3~4개월 (OpenClaw 성장 후)

OpenClaw 약점 해결

보안: 샌드박싱으로 파일 접근 범위 제한
품질: zero-configuration, 즉시 사용
신뢰: Anthropic 공식 보안 체계 적용
OpenClaw의 CVE 취약점 구조적 해결

개발사 교훈

"3시간 걸리던 작업 20분에 완료"
"회의 중 폰으로 배포를 트리거했다"
OpenClaw: 아이디어 → 1시간 → 세상을 바꿨다
Cowork: 아이디어 → 1.5주(3~4개월) → 보안·품질 확보

실제 사례 ④ OpenAI Codex — 소프트웨어를 만드는 방식이 변화

"회의 사이사이에 Codex에 태스크를 보냈다. 폰으로 PR을 리뷰하고 병합했다."

— Allison August, OpenAI 솔루션 엔지니어링 리더

+70%

주간 PR 병합 증가
(같은 팀 인원)

85%

SWE-Bench 정확도
(8회 재시도 후)

-50%

Cisco 코드 리뷰
시간 단축

-67%

Duolingo PR 리뷰
주기 단축

Cisco

코드 리뷰 시간 50% 단축

반복적·범위 명확한 리뷰에 Codex 적용
사람은 아키텍처·보안 리뷰에 집중

Temporal

Java SDK 테스트 커버리지 자동 확대

"오래된 SDK 결함 패턴"을 사전 정의
→ Codex 적용 ← Harness 방법론과 동일

Duolingo

PR 처리량 70% 증가

PR 리뷰 주기 67% 단축
개발자는 창의적 기능 설계에 집중

국내 적용 가능성과 긴급성 — 한국 개발 현장에서 더 중요한 이유

① 인력 부족 + 납기 압박

IT 개발자 '공급 < 수요' 격차 지속
공공 SI: 납기 변경 불가
→ Harness로 SIT 50% 단축 = 납기 준수
→ Claude Code로 팀 규모 보완

② 레거시 전환 폭발

코볼·PowerBuilder·Java EE 레거시 대량 존재
→ Stripe 4일 사례: 수작업 10주 → 4일
→ 국내 SI 기업의 새로운 비즈니스 모델

③ 도메인 지식 유출 방지

금융·제조·물류 전문 개발자 이직률 높음
→ Harness Catalog = 도메인 지식을
코드베이스에 영구 문서화
→ "사람이 떠나도 지식이 남는다"

④ 적용 가능 분야

제조업 MES: SIT 단축 + 도메인 버그 차단
금융 배치 시스템: 레거시 마이그레이션
공공 정보화 사업: 납기 리스크 감소
스타트업 MVP: 최소 인원 최대 속도

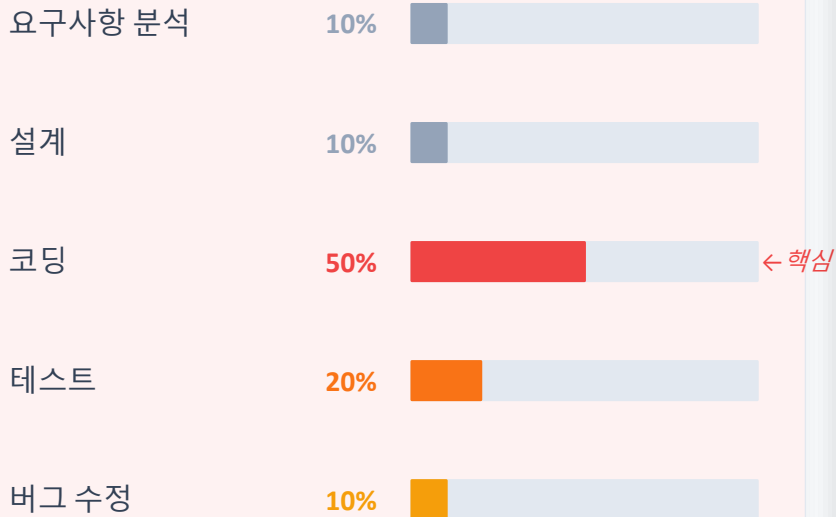
PART 7

개발자 역할 재정의

초급부터 특급까지 — 모두가 바뀐다

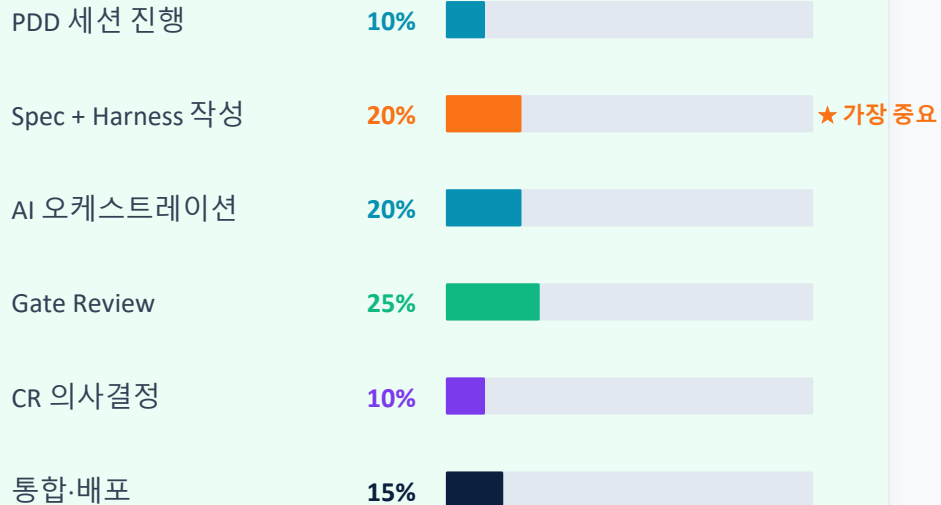
개발자의 역할 변화 — 코더 → 오케스트레이터

Before — AI 이전



역할: 코드를 직접 타이핑하는 사람

After — 4축 방법론 시대



역할: 무엇을 만들지 결정하고, 잘 만들었는지 판단하는 사람

급별 개발자의 새로운 역할 — 초급부터 특급까지 모두 변경

급별	기존 역할	새로운 핵심 역할	성장 기회
초급 개발자	선배가 시키는 기능 구현	Claude Code 세션 운영 Spec 초안 작성 Harness 테스트 결과 검증	기존보다 10배 많은 코드를 접하고 이해 → 빠른 성장
중급 개발자	복잡한 기능 설계 + 구현	Harness Catalog 도메인 항목 작성 Gate Review 검토자 Feature Flag 전략 설계	도메인 전문가로 성장 속도 가속화
고급/특급 개발자	아키텍처 설계 + 어려운 버그 디버깅	SDD Spec 표준 수립 Harness Catalog 총괄 Agent Teams 오케스트레이션 전략 Gate Review 최종 결정권자	개인 생산성이 팀 전체 생산성으로 증폭됨
PM/기획자	기획서 작성 + 개발팀 조율	PDD 세션 직접 진행 CR 범위 판단 Feature Flag 우선순위 결정	AI 시대 PM은 기술 언어와 비즈니스 언어를 동시에 구사

지금 시작하는 3가지 액션 — 내일부터 할 수 있는 것

액션 1

이번 주

Harness Catalog 초안 만들기

1. 프로젝트에서 반복되는 버그 유형 10개를 적어라.
2. Claude Code에게: "이 도메인에서 예상 결함 패턴 30개를 ID/상황/영향/방어 형식으로 생성해줘"

→ 한 시간 내에 초안이 나온다

액션 2

이번 달

다음 신규 기능에 SDD 적용하기

1. 기능 하나를 골라라.
2. 코드 작성 전에 requirements.md를 먼저 써라 (30분).
3. Claude Code에게 그 파일을 주고 구현을 시켜봐라.

→ 첫 번째 결과물의 품질 차이를 직접 느껴라

액션 3

다음 스프린트

Feature Flag 하나 도입하기

1. 가장 배포가 두려운 기능 하나를 Flag로 감싸라.
2. 배포 두려움이 얼마나 줄어드는지 느껴라.

→ 팀 전체가 경험하면 방법론이 자연스럽게 정착한다

사람은 판단한다. AI는 실행한다.

PDD

무엇을 만들까?

SDD

어떻게 만들까?

Harness

무엇을 피할까? ★

Feature Flag

실패하면?



Claude Code / Codex | 구현한다 · 테스트한다 · 반복한다

"AI가 우리를 대체하지 않는다. AI를 제대로 쓰지 않는 팀을 대체한다."

— Boris Cherny, Claude Code 창시자

Q & A

PDD

화면으로 요구사항 확정

SDD

코드 전에 Spec 완성

Harness

결함 사전 정의 → AI 방어 ★

Feature Flag

스위치로 배포 두려움 제거

감사합니다

PART Q & A

부록 (Appendix)

Q&A 대응용 – 본 발표 시간 외

[부록 A] AI가 생성하는 70~80% — 나머지 20~30%의 정체

핵심 원칙: AI가 완전히 처리하지 못하는 코드는 "코드가 아니라 의사결정"이다

유형 (비율)	구체적인 개발 사항	개발자 개입 이유	자동화 확장 조건
① 아키텍처·환경 설정 (5~8%)	docker-compose, DB pool 설정 CI/CD 세부 조건, nginx 설정	"이 서버의 RAM이 2GB" → 코드베이스 밖 인프라 실측값 필요	Spec에 인프라 사양 포함 시 50% → 70% 자동화 가능
② 외부 시스템 연동 (5~7%)	ZPL 프린터 명령어 미세 조정 ERP CSV 비공개 컬럼 파서	실제 기기 모델·현장 테스트 결과 필요 샘플 파일이 있어야 파서 작성 가능	실제 샘플 데이터 제공 시 80~90% 자동화 가능
③ 성능 최적화 (3~5%)	N+1 쿼리 리팩토링 인덱스 설계 최종 결정	실행 환경 실측(k6, EXPLAIN ANALYZE) 필요 "어느 것을 최적화할지" 우선순위 판단	Agent 서버 직접 접속 실현 시 90%+ 자동화 가능 (미래)
④ 통합·충돌 해소 (4~6%)	병렬 Worktree 병합 시 타입 충돌 공유 컴포넌트 인터페이스 불일치	두 컨텍스트 중 어느 기준으로 통일할지 [WH] 세션과 [QC] 세션은 서로를 모른다	Contract Interface 완비 시 50% → 80% 자동화 가능
⑤ 보안·인증 임계 (3~4%)	Admin 자기 역할 변경 금지 로직 "마지막 Admin 삭제 방지" 규칙	조직 정책·컴플라이언스 요구사항에 서 나옴 기술 문서가 아닌 보안 정책 문서에 있다	보안 정책을 Spec에 명시 시 70~85% 자동화 가능

자동화의 적: 암묵지 (머릿속 지식) | 자동화의 연료: 형식지 (Spec·Harness·CLAUDE.md에 쓰인 지식)

[부록 B] AI 시대 개발자 KPI 예시 — 코드 라인 수로 측정하던 시대 종료

❌ 코드 라인 수 ❌ 커밋 횟수 ❌ 종료 티켓 수 ❌ 개발 시간 → AI 활용할수록 낮은 점수를 받는 역인센티브 구조 → 방법론 정착 방해

1. 품질 기여도

40%

- ▶ Harness 결함 예방률 = 정의 결함 수 ÷ 예상 발생 결함 수 × 100 (목표: 80% 이상)
- ▶ Gate Review 1차 통과율 = 첫 Gate Review 통과 화면 수 ÷ 전체 (목표: 70% 이상)
- ▶ SIT 도메인 버그 잔존율 = 화면당 0.1건 이하

2. 생산성 증폭도

35%

- ▶ Spec 완성도 점수 = requirements/design/tasks.md 체크리스트 (목표: 90점 이상)
- ▶ Claude Code 1차 생성 채택률 = 소폭 수정으로 채택된 비율 (목표: 70% 이상)
- ▶ 자동화 Task 발굴 건수 (목표: 월 2건 이상) | 인일 절감 기여

3. 도메인 지식 자산화

25%

- ▶ Harness 항목 기여 건수 (목표: 분기당 3건 이상)
- ▶ CLAUDE.md / SKILL.md 개선 기여 (목표: 월 1건 이상)
- ▶ Gate Review 발견 이슈 중 도메인 규칙 위반 분류 비율 (목표: 50% 이상)

급별 가중치 조정:

초급: 35%/40%/25% 중급: 40%/35%/25% 고급: 35%/30%/35% 특급: 30%/25%/45%

"측정하는 것이 행동을 만든다."
도메인 지식 자산화를 측정하면 Harness Catalog가 쌓인다."